

Safety Aware NLP Using Gemma 2b : Group 5

CS F429 Natural Language Processing

Meet Parmar - 2023A7PS0406G

Rajvardhan Jadhav - 2025H1030030G

T Shubham - 2025H1030049G

Ujjwal Upadhyaya - 2025H1030140G

Computer Science

BITS Pilani, K. K. Birla Goa Campus

Date of Submission: April 15, 2026

Abstract

As Large Language Models (LLMs) become increasingly integrated into public-facing applications, ensuring they generate safe, non-toxic, and helpful content is paramount. This report details the methodology, training, and evaluation of a dual-model safety system leveraging the Gemma 2B architecture. The system consists of two distinct components: a Main LLM fine-tuned using Parameter-Efficient Fine-Tuning (PEFT) to naturally reject harmful prompts, and a Quantized Guard LLM trained as a regression scorer to proactively flag toxic inputs and outputs. By utilizing the Anthropic HH-RLHF dataset for generative alignment and the ToxiGen dataset for regression-based toxicity scoring, both models were successfully adapted under strict VRAM constraints. The report outlines the architectural advantages of Gemma 2B, the data preprocessing pipelines, the exact LoRA/QLoRA hyperparameter configurations, and the final evaluation metrics, culminating in an analysis of model hallucinations and programmatic heuristics to improve reliability.

Contents

1	Model Summary	2
1.1	Architecture	2
1.2	Difference from Others	2
1.3	Pretraining	3
1.4	Finetuning Arrangements	3
2	Finetuning Task and Dataset	4
2.1	Task Description	4
2.2	Dataset Structure and Format for the Main LLM	4
2.3	Dataset Structure and Format for the Guard LLM	4
3	Preprocessing and Tokenization	6
3.1	Main LLM	6
3.2	Guard LLM	6
4	Fine-Tuning Methodology and Training Curves	7
4.1	Main LLM (Generative Alignment)	7
4.1.1	Parameter-Efficient Fine-Tuning (PEFT)	7
4.1.2	Main LLM Training Curve	7
4.2	Guard LLM (Toxicity Regression)	8
4.2.1	Quantized Parameter-Efficient Fine-Tuning (QLoRA)	8
4.2.2	Custom Regression Trainer	8
4.2.3	Guard LLM Training Curve	8
5	Evaluation and Results	10
5.1	Main LLM Evaluation	10
5.2	Guard LLM Evaluation	10
6	Error Analysis and Scope for Improvement	11
6.1	Observed Errors (Roleplaying Hallucinations)	11
6.2	Scope for Improvement	11

Chapter 1

Model Summary

1.1 Architecture

Gemma is an open-weight Large Language Model (LLM) based on the foundational concepts introduced by Google Research in the "Attention Is All You Need" paper. Its primary function is autoregressive text generation, processing text token by token. The Gemma 2B model relies entirely on a decoder-only architecture.

Key structural parameters for the 2B model include:

- **Embedding Size (`d_model`):** 2048
- **Transformer Layers:** 18
- **Feedforward Hidden Dimensions:** 32,768
- **Attention Heads:** 8
- **Key-Value Heads:** 1 (Multi-Query Attention)
- **Query/Key/Value Head Size:** 256
- **Vocabulary Size:** 256,128 tokens (powered by the SentencePiece tokenizer)

1.2 Difference from Others

Gemma 2B differentiates itself from traditional architectures in several key ways to optimize for both expressive power and computational efficiency:

- **Decoder-Only Structure:** Unlike the original encoder-decoder transformer model, Gemma strictly utilizes a decoder-only structure tailored for generative tasks.
- **GeGLU Activation:** Within the feedforward network, the standard ReLU non-linearity is replaced by the GeGLU activation function (a variation of Gated Linear Unit). This divides activation into a sigmoidal part and a linear projection, extracting more intricate patterns.

- **Multi-Query Attention (MQA):** While larger models like the 7B variant use standard Multi-Head Attention (MHA), the 2B model specifically leverages MQA. By sharing the same key and value projections across different query projections, MQA significantly lowers computational memory requirements during inference while remaining highly effective.

1.3 Pretraining

The primary pretraining characteristics and objectives of the Gemma 2B model are:

- To predict text generation token-by-token based on extensive, meticulously filtered foundational web datasets.
- To process complex inputs utilizing a context length of 8192 tokens, allowing the model to handle up to approximately 6144 words simultaneously.
- To handle highly diverse and multilingual text inputs effectively by pretraining on an expansive 256,128 vocabulary size.

1.4 Finetuning Arrangements

Gemma is designed with flexibility in mind, distributed as both raw, pretrained base models and instruction-tuned variants. Because of its open-weight nature and efficient architectural parameters, the model easily lends itself to custom extensions and finetuning arrangements (like LoRA and QLoRA), allowing developers to adapt the model to specialized downstream tasks such as safety alignment and toxicity regression scoring.

Chapter 2

Finetuning Task and Dataset

2.1 Task Description

The primary objective of this project was to establish a dual-layer safety alignment system using the base Gemma 2B model:

1. **Main LLM (Generative Safety):** Base LLMs are pattern-matching engines trained on vast internet corpora, meaning they can inadvertently generate harmful outputs if prompted maliciously. The main fine-tuning task aimed to adjust the model's weights via Supervised Fine-Tuning (SFT) so that it consistently refuses harmful instructions while maintaining a polite tone.
2. **Guard LLM (Toxicity Scorer):** To supplement the Main LLM, a secondary Guard LLM was trained to act as a toxicity regression scorer. This model guards the Main LLM by analyzing the prompt and the output, calculating a continuous toxicity score. If the score exceeds a safe threshold, the system can fall back to a hardcoded refusal message.

2.2 Dataset Structure and Format for the Main LLM

The dataset for the Main LLM focuses on conversational safety alignment, sourced from the Anthropic/hh-rlhf dataset.

- **Format:** The dataset is structured using a plain-text conversational template where the chosen (safe/helpful) responses were isolated. The text is formatted as: `User: [Prompt] \nAssistant: [Safe Refusal]`.
- **Size:** A 10,000-example subset was extracted and split into training, validation, and test sets using an 80:10:10 ratio (8000 training, 1000 validation, and 1000 test instances).

2.3 Dataset Structure and Format for the Guard LLM

The Guard LLM was trained on the `toxigen/toxigen-data` dataset, transitioning the task from text generation to continuous score regression.

- **Format:** The dataset contains raw text and an associated `toxicity_ai` score (ranging from 1 to 5). To prepare it for Mean Squared Error (MSE) regression modeling, the raw scores were normalized to a continuous label format between 0.0 and 1.0 using the formula: $(\text{raw_score} - 1.0) / 4.0$.
- **Size:** The dataset was similarly split into training, validation, and testing sets via an 80:10:10 ratio, providing the regression head with a robust variety of toxic and benign textual examples.

Chapter 3

Preprocessing and Tokenization

3.1 Main LLM

Before passing the raw dialogue to the model, it was preprocessed using Gemma’s native tokenizer. Critical adjustments were made to ensure training stability:

- **Padding:** Sequences in a batch must be of equal length. The tokenizer was configured to use the End-Of-Sequence (EOS) token as the padding token (`pad_token_id = tokenizer.eos_token_id`). Padding was applied to the right side of the sequences.
- **Truncation and Max Length:** To manage memory constraints, sequences were truncated to a `max_length` of 256 tokens.
- **Data Type Casting:** Due to the hardware constraints of the Google Colab T4 GPU (which does not natively support `bfloat16` efficiently without scaling issues), the entire model was loaded in pure `float16` prior to training. This prevented gradient scaler crashes and ensured stable forward passes.

3.2 Guard LLM

The preprocessing pipeline for the Guard LLM required different modifications, primarily because its objective was sequence regression rather than autoregressive generation.

- **Tokenization and Padding:** The tokenizer padding was set to the right side, utilizing the EOS token. Crucially, a `DataCollatorWithPadding` was implemented to dynamically pad batches, optimizing training speed.
- **Label Mapping:** The normalized toxicity scores were strictly mapped to a feature column explicitly named `labels`. This ensures the Hugging Face `Trainer` automatically routes the continuous values to the custom MSE regression loss function.
- **Truncation:** The sequence `max_length` was truncated to 512 tokens to capture maximum context while predicting toxicity.

Chapter 4

Fine-Tuning Methodology and Training Curves

4.1 Main LLM (Generative Alignment)

4.1.1 Parameter-Efficient Fine-Tuning (PEFT)

To fine-tune the 2-billion parameter model within a 15GB VRAM limit, Low-Rank Adaptation (LoRA) was utilized. Instead of updating all model weights, LoRA freezes the base model and injects trainable rank decomposition matrices into specific attention layers.

- **Rank (r):** 8
- **Alpha (α):** 16
- **Target Modules:** q_proj, v_proj, k_proj, o_proj
- **Dropout:** 0.05

Training was executed using the `SFTTrainer` from the TRL library for 1 epoch, utilizing a learning rate of $2e-4$, a batch size of 4, and gradient accumulation steps of 4. A cosine learning rate scheduler and early stopping (patience of 3) were also employed.

4.1.2 Main LLM Training Curve

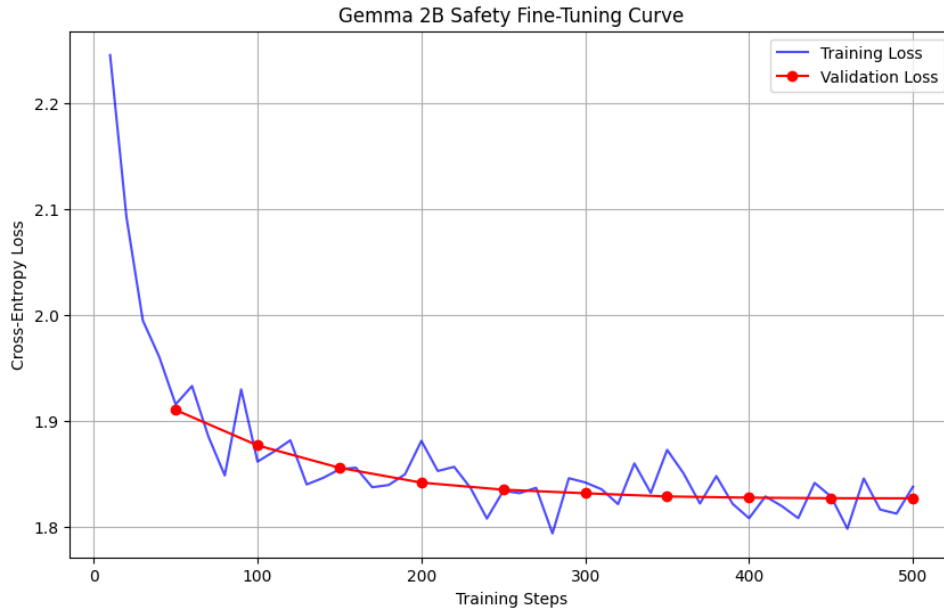


Figure 4.1: Cross-Entropy Loss over time for the Main LLM.

4.2 Guard LLM (Toxicity Regression)

4.2.1 Quantized Parameter-Efficient Fine-Tuning (QLoRA)

Unlike the main generative model, the Guard LLM utilized a 4-bit Quantized Low-Rank Adaptation (QLoRA) approach. The base model was loaded in 4-bit precision (NF4 format) with a `float16` compute data type to drastically reduce the VRAM footprint, allowing a regression head to be trained alongside the LoRA adapters.

- **Rank (r):** 16
- **Alpha (α):** 32
- **Target Modules:** `q_proj`, `k_proj`, `v_proj`, `o_proj`
- **Modules to Save:** `score` (The regression classification head)
- **Task Type:** `SEQ_CLS` (Sequence Classification configured for regression)

4.2.2 Custom Regression Trainer

A custom `RegressionTrainer` was built by subclassing the Hugging Face Trainer. This custom trainer extracted the 1D model logits and computed the Mean Squared Error (MSE) against the continuous float `labels`. The best-performing hyperparameter search yielded a learning rate of `5e-5` and a LoRA dropout of 0.1 over a single epoch.

4.2.3 Guard LLM Training Curve

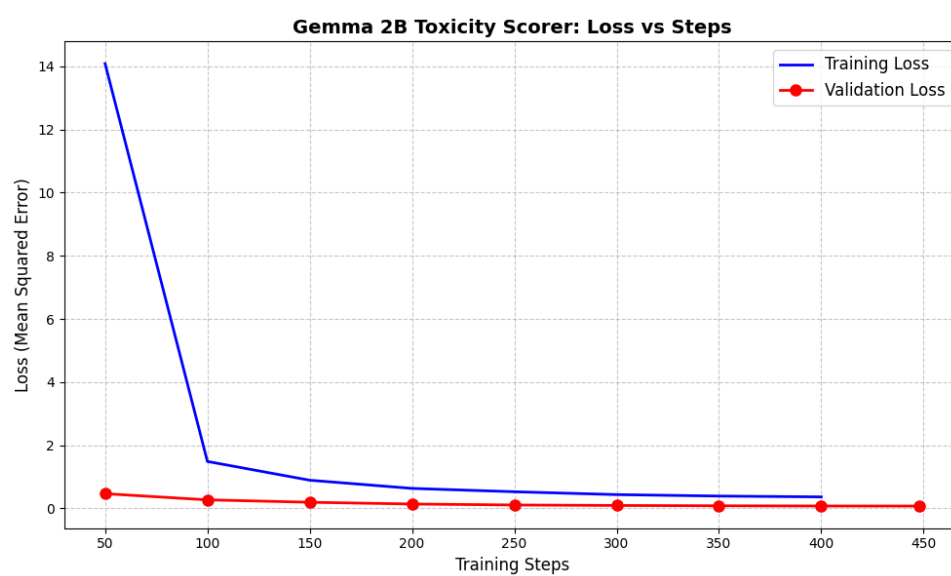


Figure 4.2: Mean Squared Error (MSE) Loss over time for the Guard LLM.

Chapter 5

Evaluation and Results

To evaluate the models’ true generalization capabilities, both were evaluated against their respective isolated, tokenized test sets.

5.1 Main LLM Evaluation

Performance was measured using Cross-Entropy Loss and Perplexity on the 1,000-sample test set.

Metric	Value
Test Loss	<i>1.8205</i>
Test Perplexity	<i>6.1751</i>

Table 5.1: Generative Metrics for the Main LLM

Analysis: A low single-digit perplexity score indicates high success for a 2B parameter model. It proves the model deeply internalized the syntax, grammar, and tone required to generate safe refusals without becoming ”confused” during next-token prediction.

5.2 Guard LLM Evaluation

Because the Guard LLM functions as a toxicity scorer, it was evaluated using regression metrics: Mean Absolute Error (MAE) and Mean Squared Error (MSE).

Metric	Value
Test MSE	<i>0.1097</i>
Test MAE	<i>0.2493</i>

Table 5.2: Regression Metrics for the Guard LLM

Analysis: The low MSE indicates that the Guard LLM’s predicted toxicity scores closely align with the normalized ground-truth ToxiGen dataset values, making it a reliable programmatic firewall for filtering inputs.

Chapter 6

Error Analysis and Scope for Improvement

6.1 Observed Errors (Roleplaying Hallucinations)

During manual inference testing of the Main LLM, a notable error occurred: after successfully generating a safe response, the model occasionally hallucinated a follow-up prompt on behalf of the user (e.g., automatically generating "User: I'm not asking for advice..."). This happens because the model acts as an autocomplete engine; without a strict stopping criterion, it continues the text pattern to satisfy the `max_new_tokens` limit.

6.2 Scope for Improvement

To improve the system's reliability, the following heuristics and updates were identified and partially implemented:

- **Programmatic Cut-Offs (Implemented):** A programmatic string-splitting heuristic was applied during generation. By executing `generated_text.split("User:")[0].strip()`, the system actively intercepts and removes any hallucinated dialogue before it reaches the end user.
- **Implementation of Chat Templates:** Instead of using raw text like `User:` and `Assistant:`, future iterations should reformat the dataset to use Gemma's native special tokens (`<start_of_turn>` and `<end_of_turn>`). This trains the model to natively output a definitive stopping token when its turn is complete.
- **Dynamic Thresholding for the Guard LLM:** The current Guard LLM acts as a continuous scorer. A future heuristic would involve dynamically shifting the "unsafe" threshold score based on the application's required sensitivity (e.g., lowering the threshold for child-facing applications to catch milder toxicity).